

HW7.1

الف) مرتب سازی پایدار (Stable Sort)

تعریف: مرتب سازی پایدار یعنی اگر دو عنصر در آرایه ورودی مقدار کلید یکسانی داشته باشند، ترتیب نسبی آنها بعد از مرتب سازی نسبت به ترتیب اولیه شان حفظ شود. به عبارت دیگر اگر عنصر A قبل از عنصر B بوده و هر دو مقدار برابر دارند، بعد از مرتب سازی هم A باید قبل از B بیاید.

چه زمانی ضروری است: وقتی داده ها بر اساس چند کلید مرتب می شوند یا اطلاعات دیگری غیر از کلید مرتب سازی برای ما مهم است. مثلاً وقتی می خواهیم لیستی را اول بر اساس یک ویژگی و سپس بر اساس ویژگی دیگر مرتب کنیم، باید مرتب سازی اول پایدار باشد تا ترتیب حاصل از آن خراب نشود.

مثال: فرض کنید لیستی از دانشجویان داریم که هر کدام نام و نمره دارند. اگر این لیست را بر اساس نام مرتب کرده باشیم و حالا بخواهیم بر اساس نمره هم مرتب کنیم، با مرتب سازی پایدار، دانشجویانی که نمره یکسان دارند همچنان به ترتیب الفبایی نام در کنار هم باقی می مانند.

ب) مرتب سازی درجا (In place) و غیردرجا (Out of place)

تفاوت: در مرتب سازی درجا، الگوریتم آرایه ورودی را مستقیماً تغییر می دهد و به حافظه اضافی (غیر از چند متغیر کمکی) نیاز ندارد. در مرتب سازی غیردرجا، الگوریتم به یک آرایه یا ساختار داده کمکی جدید نیاز دارد تا داده های مرتب شده را در آن ذخیره کند، در نتیجه حافظه بیشتری مصرف می شود.

مثال واقعی:

- مرتب سازی درجا: الگوریتم Quick Sort که عناصر را در همان آرایه اصلی جابجا می کند.
- مرتب سازی غیردرجا: الگوریتم Merge Sort که برای ادغام زیرآرایه ها به آرایه های کمکی جدید نیاز دارد.

ج) تحلیل ادعای یک الگوریتم مرتب سازی

بررسی ادعا: این ادعا از نظر عمومی درست نیست. طبق نظریه مرتب سازی مبتنی بر مقایسه، هر الگوریتمی که مرتب سازی را فقط با مقایسه زوج عناصر انجام دهد، در بدترین حالت به حداقل $O(n \log n)$ مقایسه نیاز دارد. این نتیجه از تحلیل درخت تصمیم به دست می آید: تعداد حالت های ممکن برای مرتب کردن n عنصر برابر $n!$ است و برای تشخیص بین این حالت ها با مقایسه های دودویی، حداقل $\log_2(n!)$ مقایسه لازم است که از مرتبه $n \log n$ است.

نتیجه گیری: اگر الگوریتمی واقعا با $O(n)$ مقایسه کار کند، باید فرض های اضافی داشته باشد، مثلاً اینکه داده ها از نوع خاصی هستند (مثل اعداد صحیح در یک بازه محدود) و از روش هایی مثل Counting Sort یا Radix Sort استفاده می کند که مرتب سازی مبتنی بر مقایسه نیستند. در حالت کلی و بدون فرض خاص روی داده ها، چنین ادعایی از نظر تئوری نادرست است.

د) مقایسه Quick Sort و Merge Sort

با اینکه Quick Sort در بدترین حالت مرتبه زمانی $O(n^2)$ دارد، به دلایل زیر بیشتر ترجیح داده می شود:

۱. کارایی عملی بهتر: ضرایب ثابت در عملیات Quick Sort معمولا کوچک تر است و در عمل سریع تر از Merge Sort اجرا می شود، چون عملیات کمتری روی حافظه انجام می دهد.
۲. عدم نیاز به حافظه اضافی Quick Sort: یک الگوریتم درجا است و نیازی به آرایه کمکی ندارد، در حالی که Merge Sort به حافظه اضافی به اندازه آرایه ورودی نیاز دارد.
۳. رفتار بهتر با حافظه نهان (Cache): به دلیل دسترسی محلی به داده ها در Quick Sort ، استفاده از حافظه کش پردازنده بهینه تر است که باعث افزایش سرعت اجرا می شود.
۴. کاهش احتمال بدترین حالت: با انتخاب هوشمند Pivot (مثلا به صورت تصادفی یا با روش میانه سه عدد)، احتمال رخ دادن حالت $O(n^2)$ در عمل بسیار کم می شود.